



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército

FONDO
DOCENTE
CONSEJO
FACULTAD
INGENIERIAS

FoDAMI

**IV CONGRESO ARGENTINO
DE INGENIERÍA FERROVIARIA**

**IX CONGRESO ARGENTINO
DE INGENIERÍA MECÁNICA**



Introducción temprana de la mecánica computacional en cursos de electrostática y mecánica analítica

INTEGRANTES

- Víctor A. Bettachini, Dep. de Ingeniería e Investigaciones Tecnológicas, UNLaM
- Mariano A. Real, Inst. de Calidad Industrial, UNSaM – INTI
- Edgardo Palazzo, Dep. de Materias Básicas, UTN–FRA

SÍNTESIS

- Garca

Temario

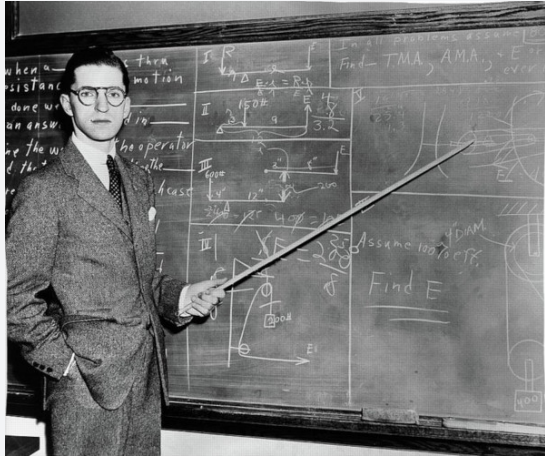
- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica
- 4 UTN-FRA — Física II
- 5 Para concluir

Temario

- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica
- 4 UTN-FRA — Física II
- 5 Para concluir

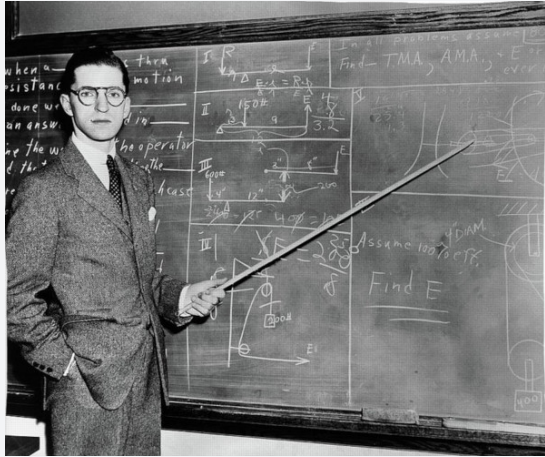
Herramientas usuales

Inicio s. XIX: pizarrón y papel

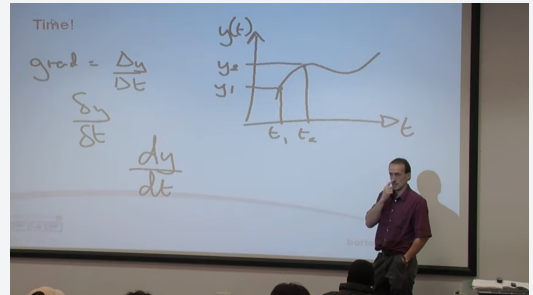


Herramientas usuales

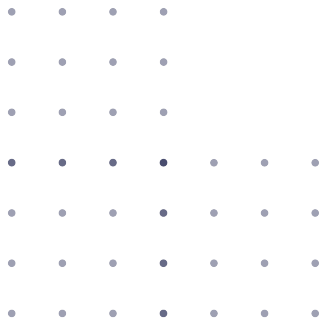
Inicio s. XIX: pizarrón y papel



Fin s. XX: computadora = ¿proyector?



Subutilización de la tecnología de la información



Subutilización de la tecnología de la información

La resistencia al cambio



Subutilización de la tecnología de la información

La resistencia al cambio



Décadas de atraso

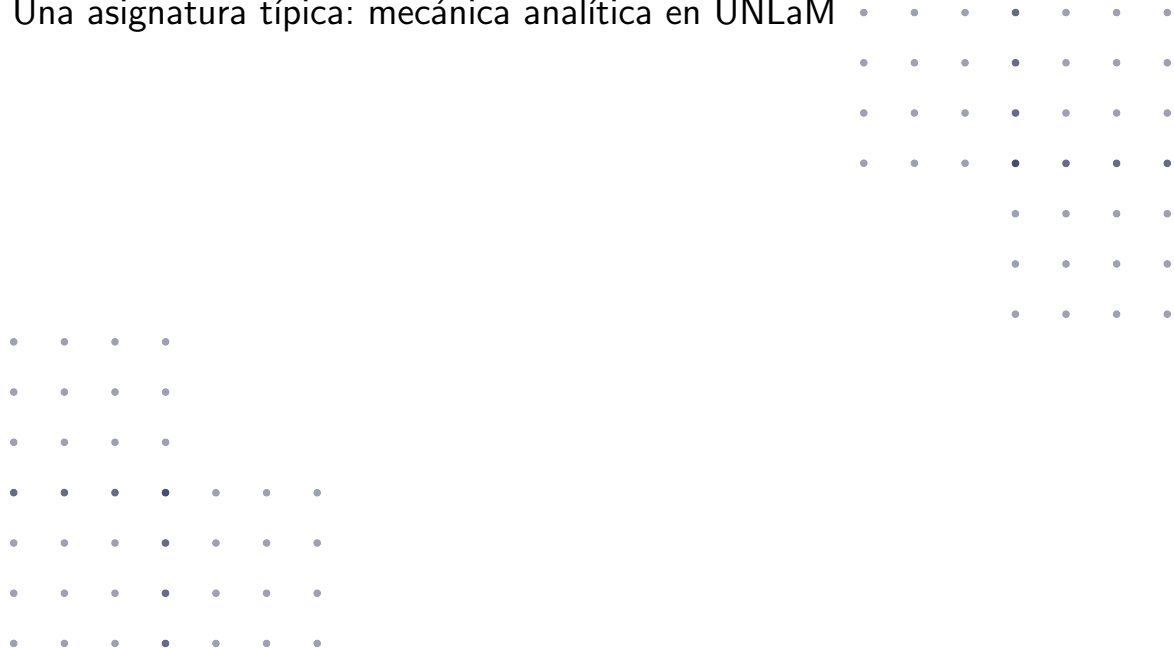


¿El miedo a la adopción tecnológica es lo único que no ha evolucionado?

Temario

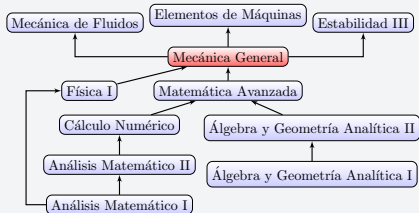
- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica
- 4 UTN-FRA — Física II
- 5 Para concluir

Una asignatura típica: mecánica analítica en UNLaM



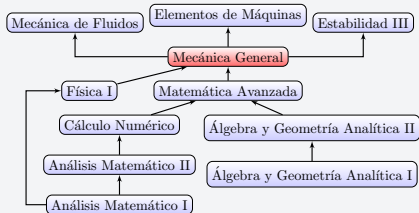
Una asignatura típica: mecánica analítica en UNLaM

Aprobaron álgebras y análisis



Una asignatura típica: mecánica analítica en UNLaM

Aprobaron álgebras y análisis



¿Por qué no automatizarlos?

```
[14]: sistemaEcuaciones = [  
      x_EL,  
      phi_EL,  
    ]  
variablesDespeje = [x.diff(t,2), phi.diff(t,2)] # despejar aceleraciones generalizadas  
variablesDespeje_sol = sym.nonlinsolve(sistemaEcuaciones, variablesDespeje ).args[0]  
  
[15]: x_pp = sym.Eq(variablesDespeje[0], variablesDespeje_sol.args[0]) # [m s-2]  
phi_pp = sym.Eq(variablesDespeje[1], variablesDespeje_sol.args[1]) # [m s-2]  
x_pp, phi_pp
```

$$[15]: \left(\begin{array}{l} \ddot{x} = \frac{-\ell m_2 \sin(\phi) + \frac{\ell m_2 (\ell m_2 \cos(\phi) \dot{\phi}^2 + g m_1 + g m_2) \sin(\phi)}{m_1 + m_2 \sin^2(\phi)}}{\ell m_2 \cos(\phi)}, \ddot{\phi} = -\frac{(\ell m_2 \cos(\phi) \dot{\phi}^2 + g m_1 + g m_2) \sin(\phi)}{\ell (m_1 + m_2 \sin^2(\phi))} \end{array} \right)$$

Cuaderno Jupyter: teoría + código ejecutable

File Edit View Run Kernel Tabs Settings Help

PÉNDULOENHEBRADOSOLVED.IPYNB

<> M ≡

Péndulo enhebrado en aro

Enunciado

1. Construya el Lagrangiano y la(s) función(es) de ligadura

2. Calcule las ecuaciones de Euler-Lagrange

3. Obtenga una expresión para la tensión que ejerce la barra

4. Grafique la tensión en los primeros diez segundos de la dinámica.

Launcher x cursoJupyter.ipynb x pénduloEnhebradoSolved.ipynb x

Python 3

3. Obtenga una expresión para la tensión que ejerce la barra

$$Q_d = \lambda_1 \frac{\partial f_1}{\partial d} = \lambda_1$$

Por tanto hay que resolver el sistema con las 3 ecuaciones de Euler-Lagrange y la única de ligadura para determinar λ_1 . Esta última hay que resolverla para su caso homogéneo y expensar su derivada segunda para que esté en el mismo orden que las de Euler-Lagrange, a fin de cuentas estamos resolviendo sistemas diferenciales de 2.º orden.

```
[14]: f_1
```

```
[14]: f_1 = -l + d
```

Determinamos también $\ddot{\theta}_1$ y $\ddot{\theta}_2$ pues serán necesarias para los cálculos numéricos posteriores.

```
[15]: sistema = [theta1_EL.expand(),
               theta2_EL.expand(),
               d_EL.expand(),
               sym.Eq(f_1.rhs.diff(t,2), 0)], # esto es igual a d punto punto = 0
          ]
variables = [theta1.diff(t,2), theta2.diff(t,2), lambda_1]
variables_sol = sym.nonlinsolve(sistema, variables).args[0]
```

```
[16]: lambda_1_sol = sym.Eq(lambda_1, variables_sol.args[2])
      lambda_1_sol.simplify()
```

```
[16]:
```

$$\lambda_1 = \frac{m \left(2a \cos(\theta_1 - \theta_2) \dot{\theta}_1^2 + g \cos(2\theta_1 - \theta_2) + g \cos(\theta_2) + 2d \ddot{\theta}_2 - 2\ddot{d} \right)}{\cos(2\theta_1 - 2\theta_2) - 3}$$

Simple 0 2 Python 3 | Idle

Saving completed

Mode: Command Ln 1, Col 1 pénduloEnhebradoSolved.ipynb

Google Colaboratory: cuadernos Jupyter en la nube

- Profesor puede editar y comentar el trabajo del alumno.
- Múltiples estudiantes pueden trabajar el mismo cuaderno en forma remota.

The screenshot shows a Google Colaboratory notebook titled "07 No conservativas | ej4". The interface includes a top bar with navigation links (Archivo, Editar, Ver, Insertar, Entorno de ejecución, Herramientas, Ayuda) and a status bar indicating the last edit was 3 days ago. The notebook content is divided into two main sections: "Energía potencial" and "Lagrangiano".

Energía potencial

```
[ ] # Energía potencial
m1_V = - (m1* g* (- N.y)).dot(m1_r)
# pot_k1 = unMedio* ( -k1* ((l10 + x1)* (sym.cos(theta) - sym.sin(theta)) )**2 ) # mal
pot_k1 = unMedio* k1* (l10 + x1)**2 # Lo escribí yo
# pot_k2 = unMedio* -k2* (l20 + x)**2
pot_k2 = unMedio* k2* (l20 + x)**2
V = sym.Eq(sym.Symbol('V'), m1_V + pot_k1 + pot_k2 ) #agrega el potencial elastico k en la ecuacion
V
```

$$V = gm_1(-l_{10} - x_1) \sin(\theta) + \frac{k_1(l_{10} + x_1)^2}{2} + \frac{k_2(l_{20} + x)^2}{2}$$

Lagrangiano

```
[ ] L = sym.Eq(sym.Symbol('\mathcal{L}'), (T.rhs - V.rhs))
L
```

$$\mathcal{L} = -gm_1(-l_{10} - x_1) \sin(\theta) - \frac{k_1(l_{10} + x_1)^2}{2} - \frac{k_2(l_{20} + x)^2}{2} + \frac{(m_0 + m_1)(2 \cos(\theta) \dot{x}_1 + \dot{x}^2 + \dot{x}_1^2)}{2}$$

ECUACIONES DE EULER

Para x

Comments on the right side of the notebook:

- Victor Alexis Bettachini (31 de may. de 2021):
(editado el 31 de may. de 2021)
- El estiramiento del resorte de k_1 es colineal con x_1 . No tienen sentido pensar en proyecciones (si es lo que hiciste, que realmente no entiendo).
- ¿Porque negativos los k?
- Victor Alexis Bettachini (31 de may. de 2021)

Repositorio GitHub

- Organizado por tema, acceso libre, fácil de actualizar.
- Google Colaboratory carga cuadernos Jupyter directamente desde GitHub.

The screenshot shows the GitHub repository page for 'MecanicaAnaliticaComputacional' by user 'bettachini'. The repository is public and has 2 branches and 1 tag. A warning message states 'Your master branch isn't protected'. A merge pull request #27 from 'realmariano/master' is shown. The file list includes folders for '00Generalidades', '01Vectorial', '03Energia', '04EulerLagrange', '05Ligaduras', '06Simulación', '07FuerzasLigadura', '08NoConservativas', '09aNoInercial', '09bTensorInercia', '10RotaciónEuler', '11Vibraciones1GdL', '12VibracionesNGdL', '77Notas', '88presentación', and 'JupyterPythonLaTeX'. The right sidebar shows the 'About' section with a description of the repository, tags for 'python', 'physics', 'mechanical-engineering', 'ingeniería', 'física', 'mecánica', 'lagrangian', 'vibrations', and 'mechanics-of-solids', and a 'Releases' section for 'v2023p'.

MecanicaAnaliticaComputacional Public

2 branches 1 tag

Your master branch isn't protected

Merge pull request #27 from realmariano/master

File	Description	Time
00Generalidades	Correcciones LaTeX	3 months ago
01Vectorial	re-escritura para que LaTeX en líneas	3 months ago
03Energia	Correcciones LaTeX	3 months ago
04EulerLagrange	Modificada la definición de T y V para que devuelva ecuaciones y ent...	2 months ago
05Ligaduras	ligadura atwood replace 2	3 months ago
06Simulación	06Simulación solved	last month
07FuerzasLigadura	Atwood compuesta: coordenadas poetas en figura	5 months ago
08NoConservativas	no conservativas cilindros solidarios	last month
09aNoInercial	copias anafía	last month
09bTensorInercia	tensorInercia 2	3 weeks ago
10RotaciónEuler	e4 enunciado	last month
11Vibraciones1GdL	guias mac	last year
12VibracionesNGdL	guias mac	last year
77Notas	typo	last month
88presentación	sugerencias muy mínimas de modificación	4 hours ago
JupyterPythonLaTeX	01Vectorial este	3 months ago

About

Repositorio de la asignatura "Mecánica Analítica Computacional" de la carrera de grado en ingeniería mecánica de la Universidad Nacional de La Matanza.

python physics mechanical-engineering ingeniería física mecánica lagrangian vibrations mechanics-of-solids

Releases

v2023p (Latest) on Aug 10

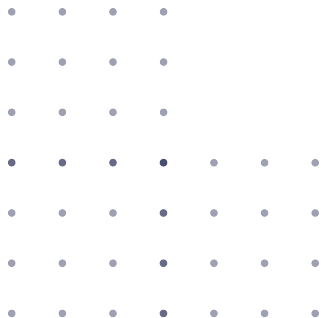
Packages

No packages published

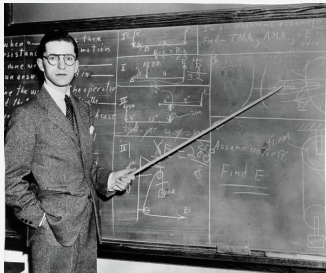
Contributors

bettachini Victor Alexis Bettachini

Valorar el tiempo de docentes y alumnos

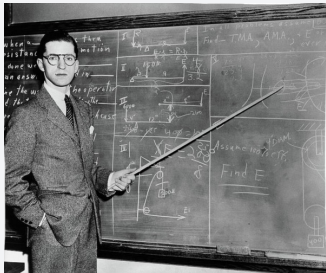


Valorar el tiempo de docentes y alumnos



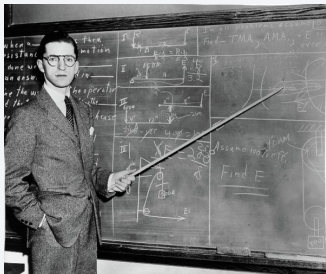
- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación

Valorar el tiempo de docentes y alumnos



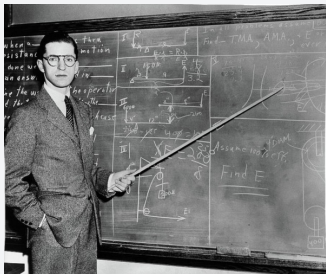
- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno

Valorar el tiempo de docentes y alumnos



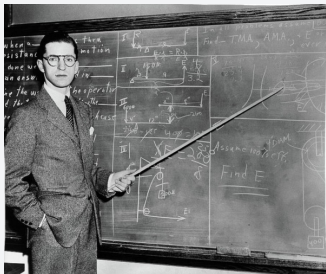
- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .

Valorar el tiempo de docentes y alumnos



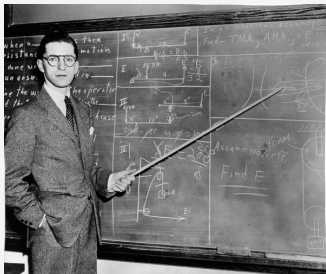
- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

Valorar el tiempo de docentes y alumnos



- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

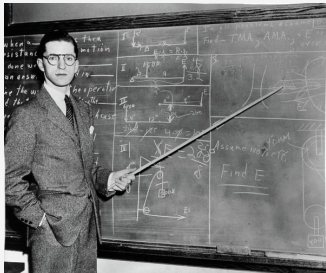
Valorar el tiempo de docentes y alumnos



- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

Licklider (1957): 85 % de “pensar” es lo “mundano”: organizar información, calcular, dibujar...

Valorar el tiempo de docentes y alumnos



- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

Licklider.(1957): 85 % de “pensar” es lo “mundano”: organizar información, calcular, dibujar...

```
> Code < Text
[ ]  Pizarrón
-----
V : [symbol type] (sympy core relational equality)
de la izquierda equilibria la energía cinética del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
V : [symbol type] (sympy core relational equality)
de la izquierda equilibria la energía potencial del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
conservadadenergiatotal: [symbol type] (sympy core symbol type)
Para la que quiere obtener la ecuación de Euler-Lagrange

Retorno
-----
[symbol type] (sympy core relational equality)
Ecuación de Euler-Lagrange homogénea para la coordenadageneralizada

lagrangiano = (T - V).subs(V, Vdiff)
s = sym.symbols('t') # como se decide respecto al tiempo con la función diff se decide si como símbolo
diff(s, t)
- lagrangiano.diff(coordadadeneralizada)
- lagrangiano.diff(coordadadeneralizada.diff(t)).diff(t)
s
simplify()

[ ]  theta_RL = solve(lagrange(f, s, theta)
theta_RL
-----

$$\left( \frac{m_1 m_2 \sin^2(\theta)}{2} - m_1 m_2 \cos^2(\theta) - m_1 m_2 \sin^2(\theta) \left( \frac{m_1 m_2 \sin^2(\theta)}{2} + m_1 m_2 \cos^2(\theta) \right) \right) = 0$$

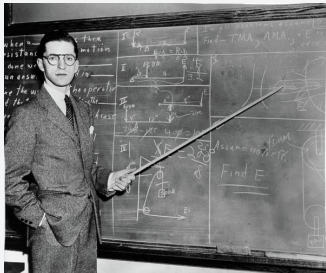

[ ]  theta_RL = sym.Rg(theta.diff(t, 2), sym.solve(theta_RL, theta.diff(t, 2))) # [0] # [0] toma el único elemento de la lista
theta_RL
-----

$$\theta = \frac{(m_1 m_2 \sin^2(\theta) - m_1 m_2 \cos^2(\theta) + m_1 m_2 \sin^2(\theta)) \sin(\theta)}{2 (m_1 m_2 \cos^2(\theta) + m_1 m_2 \sin^2(\theta))}$$

```

- Profesor: ideas $\xrightarrow{\text{actualiza}}$ código/teoría en repositorio

Valorar el tiempo de docentes y alumnos



- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

Licklider.(1957): 85 % de “pensar” es lo “mundano”: organizar información, calcular, dibujar...

```
+ Code + Text

[ ]  Pizarrón
    ...
    v = (igualdad sympy (sympy core relational equality)
    de la cual derecha especifica la energía cinética del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
    v = (igualdad sympy (sympy core relational equality)
    de la cual derecha especifica la energía potencial del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
    coordenadas generalizadas: símbolo sympy (sympy core symbol symbol)
    Para la que quiere obtener la ecuación de Euler-Lagrange

    Retorno
    ...
    igualdad sympy (sympy core relational equality)
    ...
    ecuación de Euler-Lagrange homogénea para la coordenada generalizada

    lagrangiano = (T - V).subs({x: x, y: y, z: z})
    x = sym.Symbol('x') # como se declara respecto al tiempo con la función diff se declara a como símbolo
    return sym.lambdify(
        [coordenadas generalizadas],
        lagrangiano.diff(coordenas generalizadas).diff(t))
    )

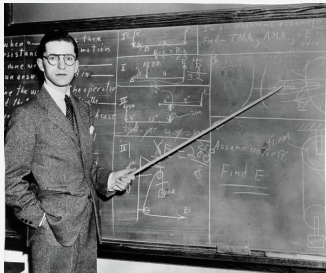
[ ]  theta_R = solve_lagrange(t, x, theta)
    theta_R =
    
$$\left( \frac{m_1 \sin(2\theta_R)}{2} - m_1 \cos(\theta_R) - m_2 \sin^2(\theta_R) - \frac{m_2 \sin(2\theta_R)}{2} + g m_1 \sin(\theta_R) \right) = 0$$


[ ]  theta_R = sym.lambdify(theta.diff(t, 2), sym.lambdify(theta_R, theta.diff(t, 2)))[0] # [0] toma el único elemento de la lista
    theta_R =
    
$$\theta = \frac{(m_1 \cos(\theta_R) - m_2 \cos(\theta_R) + g m_1) \sin(\theta_R)}{2 (m_1 \cos(\theta_R) + m_2 \sin^2(\theta_R))}$$

```

- Profesor: ideas $\xrightarrow{\text{actualiza}}$ código/teoría en repositorio
- Estudiante: repositorio del curso $\xrightarrow{\text{clona}}$ uno propio

Valorar el tiempo de docentes y alumnos



- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Aburrimiento $\implies \downarrow$ pérdida de concentración en la clase

Licklider.(1957): 85 % de “pensar” es lo “mundano”: organizar información, calcular, dibujar...

```
+ Code + Text

[ ]  pizarrón:
    ...
    * Igualdad Symy (symy core relational equality)
    de la izquierda equilibria la energía cinética del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
    * Igualdad Symy (symy core relational equality)
    de la izquierda equilibria la energía potencial del sistema en función de coordenadas y velocidades generalizadas y el tiempo.
    coordenadas generalizadas (simbol symy)
    Para la que quiere obtener la ecuación de Euler-Lagrange

    Retorno
    ...
    Igualdad Symy (symy core relational equality)
    ...
    Ecuación de Euler-Lagrange homogénea para la coordenada generalizada

    lagrangiano = (T - V).subs(V=0).simplify()
    n = sym.Symbol('n') # como se deriva respecto al tiempo con la función diff se declara a como símbolo
    return sym.Eq(
        -lagrangiano.diff(coordenada generalizada)
        + lagrangiano.diff(coordenada generalizada.diff(t)).diff(t)
    ).simplify()

[ ]  theta_R = solve_lagrange(t, n, theta)
    theta_R


$$\left( \frac{m_1 m_2 \sin^2(\theta)}{2} - m_1 m_2 \cos(\theta) - m_2 \sin^2(\theta) \left( \frac{m_1 \cos(\theta)}{2} + g \cos(\theta) \right) \right) = 0$$


[ ]  theta_R = sym.Eq(theta.diff(t,2), sym.solve(theta_R, theta.diff(t,2)))[0] # [0] toma el único elemento de la lista
    theta_R


$$\theta = \frac{(m_1 \cos(\theta) - m_2 \cos(\theta) + g \sin(\theta)) \sin(\theta)}{2(m_1 \cos(\theta) + m_2 \sin^2(\theta))}$$

```

- Profesor: ideas $\xrightarrow{\text{actualiza}}$ código/teoría en repositorio
- Estudiante: repositorio del curso $\xrightarrow{\text{clona}}$ uno propio
- **Recicla** código del profesor para resolver nuevos problemas

Temario

- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica**
- 4 UTN-FRA — Física II
- 5 Para concluir

1er clase — SymPy: álgebra y análisis simbólico

- El esfuerzo del estudiante se centra en la nueva temática

```
[8]: m2_v_cuadrado = m2_v.dot(m2_v)
m2_v_cuadrado
```

$$[8]: \ell^2 \sin^2(\varphi) \dot{\varphi}^2 + (\ell \cos(\varphi) \dot{\varphi} + \dot{x})^2$$

Con esto la energía cinética queda

$$\begin{aligned} T(\dot{x}_1, \varphi, \dot{\varphi}) &= \frac{m_1}{2} \left(\dot{r}_1 \right)^2 + \frac{m_2}{2} \left(\dot{r}_2 \right)^2 \\ &= \frac{m_1}{2} \dot{x}^2 + \frac{m_2}{2} (\dot{x}^2 + 2\dot{x}\ell \cos \varphi \dot{\varphi} + \ell^2 \dot{\varphi}^2) \end{aligned}$$

```
[9]: # Energía cinética
unMedio = sym.Rational(1,2) # Rational: fracción de enteros, alternatively podría haberse usado 0.5
m1_T = unMedio*m1*m1_v_cuadrado
m2_T = unMedio*m2*m2_v_cuadrado
T = sym.Eq(sym.Symbol('T'), (m1_T + m2_T)) # simplify: simplifica usando factor común y otras operaciones
T
```

$$[9]: T = \frac{m_1 \dot{x}^2}{2} + \frac{m_2 (\ell^2 \sin^2(\varphi) \dot{\varphi}^2 + (\ell \cos(\varphi) \dot{\varphi} + \dot{x})^2)}{2}$$

3er clase — Dinámica por Euler-Lagrange

Ecuaciones de Euler-Lagrange

Para x

```
[8]: x_EL = sym.Eq(L.rhs.diff(x) - L.rhs.diff(x.diff(t)).diff(t), 0).simplify() # ecuación igualando a cero
x_EL
```

$$[8]: m_1 \ddot{x} + m_2 \left(-\ell \sin(\phi) \dot{\phi}^2 + \ell \cos(\phi) \ddot{\phi} + \ddot{x} \right) = 0$$

Esta es una ecuación diferencial lineal de segundo orden homogénea. De aquí podría despejarse $\ddot{x}$

```
[9]: sym.Eq(x.diff(t,2),
          list( sym.solve(x_EL, x.diff(t,2) ) )[0] # solveset devuelve un set, que convertimos a lista
          ) # aceleración = x punto punto [m s-2]
```

$$[9]: \ddot{x} = \frac{\ell m_2 \left(\sin(\phi) \dot{\phi}^2 - \cos(\phi) \ddot{\phi} \right)}{m_1 + m_2}$$

Pero queda en función de otra aceleración $\ddot{\phi}$.

Para ϕ

```
[10]: phi_EL = sym.Eq(L.rhs.diff(phi) - L.rhs.diff(phi.diff(t)).diff(t), 0).simplify() # ecuación igualando a cero
phi_EL
```

5ta clase — SciPy: cálculo numérico

```
[22]: # defino una función con el sistema de derivadas
      # t : no se usa en este sistema pero lo dejamos para uso posterior
      # y : lista de estado con [y[0], y[1], y[2], y[3]]
      # y[0]: x
      # y[1]: x punto
      # y[2]: phi
      # y[3]: phi punto
      # dydt : lista de derivadas
      def y_punto(t, y):
          dydt = [y[1],
                  x_pp_numpy(y[0], y[1], y[2], y[3]),
                  y[3],
                  phi_pp_numpy(y[0], y[1], y[2], y[3]),
                  ]
          return dydt

[23]: # Integración de a pasos en el tiempo
      y_ode2 = solve_ivp(y_punto, (t_rango[0], t_rango[-1]), y_inicial, t_eval = t_rango)

[25]: y_ode2.y[0]
```

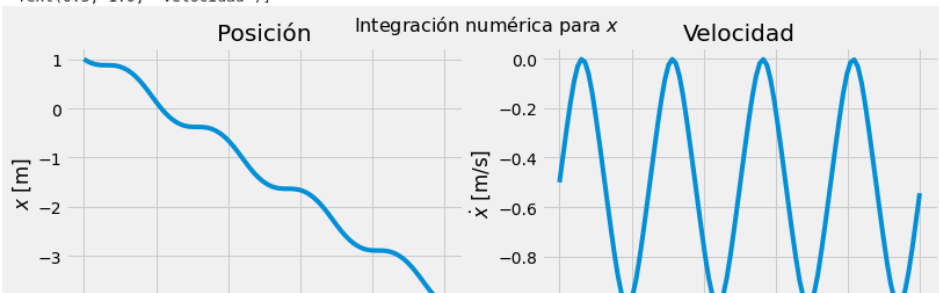
```
[25]: array([ 1.          ,  0.95510744,  0.92131146,  0.89820932,  0.88468059,
          0.87877042,  0.87745354,  0.87702754,  0.87352768,  0.86357726,
          0.84474673,  0.81565733,  0.77559949,  0.72423163,  0.66166451,
          0.588266   ,  0.50468237,  0.41250381,  0.31433661,  0.21366454,
          0.11444308,  0.02023394, -0.06599563, -0.14244216, -0.20809592,
          -0.26250272, -0.30576388, -0.33796804, -0.35953138, -0.37175469,
          -0.3760222 , -0.37701011, -0.37854158, -0.38007486, -0.38107026])
```

5ta clase — matplotlib: análisis gráfico de resultados • • • • •

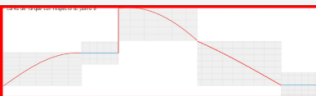
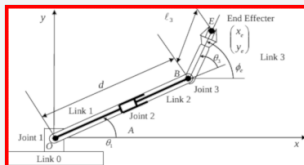
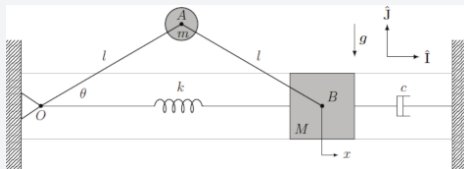
```
[26]: solucion = y_ode2
      nombreCoordenada = 'x'

      fig, ax = plt.subplots(nrows= 1, ncols= 2, squeeze=False, figsize=(12, 4)) # dos figuras en la misma fila
      fig.suptitle('Integración numérica para $'+ nombreCoordenada + '$', fontsize=16)
      ax[0,0].plot(solucion.t, solucion.y[0]) # posición x
      ax[0,0].set(xlabel='t [s]', ylabel= '$' + nombreCoordenada+ '$ [m]', title='Posición')
      ax[0,1].plot(solucion.t, solucion.y[1]) # velocidad x
      ax[0,1].set(xlabel='t [s]', ylabel='$\dot{x}$ [m/s]', title='Velocidad')
```

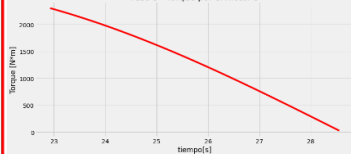
```
[26]: [Text(0.5, 0, 't [s]'),
      Text(0, 0.5, '$\dot{x}$ [m/s]'),
      Text(0.5, 1.0, 'Velocidad')]
```



7ma clase — Fuerzas no conservativas y de ligadura



Paso 5 - torque por el motor 3



Paso 6

En este paso la primera sección se extiende según los valores calculados durante el desarrollo del paso 4. L2 se extiende unos 1.0778 metros, calculamos el tiempo transcurrido para esto.

```
tp6 = l2f/v_v
tpp6 = sym.symbols("Tiempo_paso6")
sym.Eq(tpp6, tp6) #Segundos
```

Tiempo_{paso6} = 21.5571741279056

Motor 1

```
Qbr1p6 = ncl1*g #Fuerza para el cilindro 1
Qbr2p6 = ncl1*g #Fuerza para el cilindro 2
Qbr3p6 = ncl1*g #Fuerza para el cilindro 3
Qep6 = (ne+mp)*g #Fuerza para el conjunto efector-pieza
Qp6 = 3*Qbr1p6 + Qep6 #Fuerza total realizada por el primer motor motor 1

Wbr1p6 = Qbr1p6*y1.diff(12) #Torque para el cilindro 1
Wbr2p6 = Qbr2p6*y2p3.diff(12) #Torque para el cilindro 2
Wbr3p6 = Qbr3p6*y3p5.diff(12) #Torque para el cilindro 3
Wep6 = Qep6*yep5.diff(12) #Torque para el conjunto efector-pieza
Wp6 = Wbr1p6 + Wbr2p6 + Wbr3p6 + Wep6 #Torque total realizada por el motor 1
```

8va clase — GitHub Copilot: código de un LLM

- Tras dominar lo básico los estudiantes pueden aprovecharse de la IA.

```
lagrangiano = (T.rhs - V.rhs).expand()
t = sym.Symbol('t') # como se deriva respecto al tiempo con la función diff se declara t como símbolo
return sym.Eq(
    lagrangiano.diff(coordenadaGeneralizada)
    - lagrangiano.diff(coordenadaGeneralizada.diff(t)).diff(t)
    , 0
).simplify()
```

[21]

```
x1_EL = eulerLagrange(T, V, x1)
x1_EL
```

[22]

...

$$\frac{\pi^2 M \ddot{x}_1}{2} - gm_1 + gm_2 + m_1 \ddot{x}_1 + m_2 \ddot{x}_1 = 0$$

Esta es una ecuación diferencial lineal de segundo orden homogénea. De aquí se puede despejar $\ddot{x}$

▷

```
#Despejar x1PuntoPunto
x1PuntoPunto = sym.solve(x1_EL, x1.diff(t, t)).args[0]
```

El código como competencia del mecánico

Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El código como competencia del mecánico

Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El alumno “construye” su competencia

- Problemas de ejemplo se resuelven con el código provisto por el profesor

El código como competencia del mecánico

Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El alumno “construye” su competencia

- Problemas de ejemplo se resuelven con el código provisto por el profesor
- El estudiante lo **recicla** para resolver los de la guía de ejercicios

El código como competencia del mecánico

Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El alumno “construye” su competencia

- Problemas de ejemplo se resuelven con el código provisto por el profesor
- El estudiante lo **recicla** para resolver los de la guía de ejercicios
- Gradualmente adquiere autonomía y genera su propio código.

El código como competencia del mecánico

Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El alumno “construye” su competencia

- Problemas de ejemplo se resuelven con el código provisto por el profesor
- El estudiante lo **recicla** para resolver los de la guía de ejercicios
- Gradualmente adquiere autonomía y genera su propio código.
- Finalmente, puede aprovecharse de la IA para acelerar su trabajo.

El código como competencia del mecánico

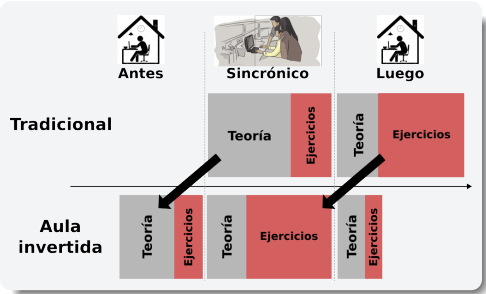
Enfoque construccionista

Papert (1980) “El mejor aprendizaje tiene lugar cuando el sujeto toma el mando”

El alumno “construye” su competencia

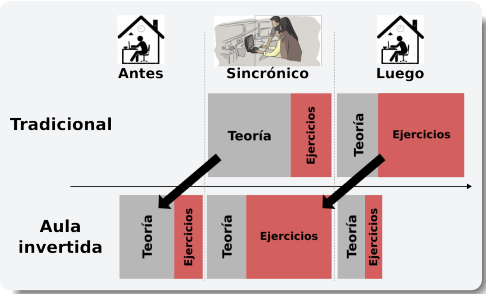
- Problemas de ejemplo se resuelven con el código provisto por el profesor
- El estudiante lo **recicla** para resolver los de la guía de ejercicios
- Gradualmente adquiere autonomía y genera su propio código.
- Finalmente, puede aprovecharse de la IA para acelerar su trabajo.
- Tras cursar conserva su repositorio con todo su código.

Clase invertida — Ciclo semanal



Sincrónico	Teoría	Ejercicios
Antes	Leer y aplicar	Iniciarles
Durante	Consultas	Completarles
Luego	Aclaraciones adicionales	Ayuda de auxiliares

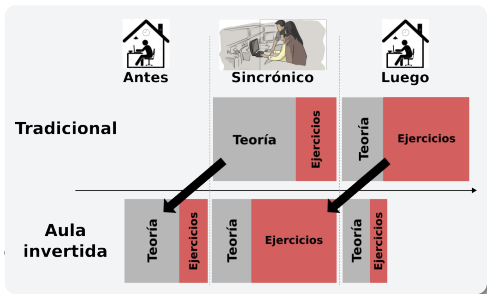
Clase invertida — Ciclo semanal



Sincrónico	Teoría	Ejercicios
Antes	Leer y aplicar	Iniciarles
Durante	Consultas	Completarles
Luego	Aclaraciones adicionales	Ayuda de auxiliares

- Publicación semanal de teoría y ejemplos en cuadernos Jupyter, y guía de ejercicios.

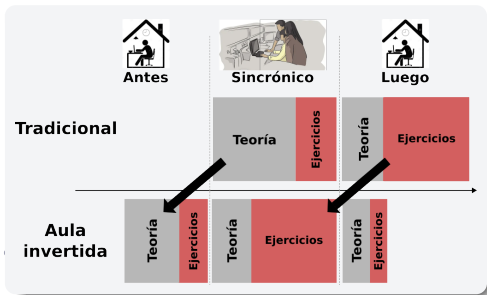
Clase invertida — Ciclo semanal



Sincrónico	Teoría	Ejercicios
Antes	Leer y aplicar	Iniciarles
Durante	Consultas	Completarles
Luego	Aclaraciones adicionales	Ayuda de auxiliares

- Publicación semanal de teoría y ejemplos en cuadernos Jupyter, y guía de ejercicios.
- Ejercicios de la semana anterior **deben** entregarse para evaluación.

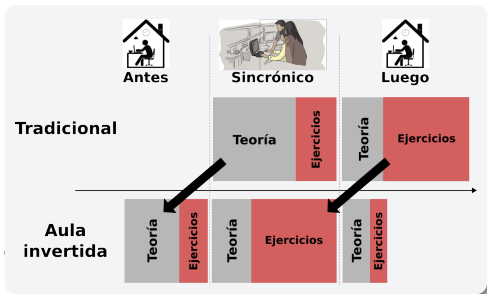
Clase invertida — Ciclo semanal



Sincrónico	Teoría	Ejercicios
Antes	Leer y aplicar	Iniciarles
Durante	Consultas	Completarles
Luego	Aclaraciones adicionales	Ayuda de auxiliares

- Publicación semanal de teoría y ejemplos en cuadernos Jupyter, y guía de ejercicios.
- Ejercicios de la semana anterior **deben** entregarse para evaluación.
- Las consultas **asincrónicas** en línea (24/7) son **públicas** para que otros las aprovechen.

Clase invertida — Ciclo semanal



Sincrónico	Teoría	Ejercicios
Antes	Leer y aplicar	Iniciarles
Durante	Consultas	Completarles
Luego	Aclaraciones adicionales	Ayuda de auxiliares

- Publicación semanal de teoría y ejemplos en cuadernos Jupyter, y guía de ejercicios.
- Ejercicios de la semana anterior **deben** entregarse para evaluación.
- Las consultas **asincrónicas** en línea (24/7) son **públicas** para que otros las aprovechen.
- Encuentros **sincrónicos** con auxiliares para finalizar ejercicios.

Microsoft Teams: seguimiento individualizado

Registro semanal de entregas de ejercicios

Q Buscar

...

MR

Calificaciones

Vencimiento el 28 sept

📖

☰

Exportar a Excel

⌵

⚙

	g06e03	g06e04	g06e05	g05e01a	g05e01c	g05e02	g05e03
Buscar alumnos	28 sept	28 sept	28 sept	14 sept	14 sept	14 sept	14 sept
📊 Promedio de clase							
👤	Visto		Visto	Devuelto	Entregado	Entregado	Entregado
👤				Devuelto	Entregado	Entregado	Entregado
👤				Entregado	Entregado	Entregado	Entregado
👤	Visto	Visto	Visto	Entregado	Entregado	Entregado	Entregado
👤				Entregado	Entregado		Entregado
👤	Visto	Visto	Visto	Entregado	Entregado	Visto	Entregado
👤	Visto			Entregado	Entregado	Entregado	Entregado

Acceso libre al material de la asignatura

Repositorio GitHub

paper	paper/10.21105.jose.00256.pdf	2 months ago
.gitignore	Open in colab	last year
LICENSE	License changed to CC.	last year
README.md	Changed expression: misaligned gear	3 weeks ago
todo.txt	todo.txt upd	2 weeks ago

README License



Computational Analytical Mechanics

2024 Victor A. Bettachini

Access to the material: <https://uniam.github.io/ComputationalAnalyticalMechanics/>

This is the repository for the course [Computational Analytical Mechanics](#), for the [mechanical engineering degree](#) at the [Department of Engineering and Research in Technology](#), Universidad Nacional de La Matanza.

It's a code-based undergraduate course on analytical mechanics for engineering students with little to no prior programming knowledge. This 16-week flipped classroom course focuses on providing skills to calculate dynamics and strains of simple mechanical devices modelled as rigid bodies by solving Euler-Lagrange equations. Problems sets with increasingly complex exercises are introduced each week. Python-based solutions to previous ones are reused by students, who through small incremental modifications over the code presented by the teaching staff, build their own library of solutions to address mechanical modelling challenges.

 **bettachini** Victor Alexis Bettachini

 **Jaime2007**

Deployments 36

 **github-pages** 2 weeks ago

[+ 35 deployments](#)

Languages

 Jupyter Notebook 99.4%  Text 0.6%

Acceso libre al material de la asignatura

Repositorio GitHub

paper	paper/10.21105.jose.00256.pdf	2 months ago
.gitignore	Open in colab	last year
LICENSE	License changed to CC	last year
README.md	Changed expression: misaligned gear	3 weeks ago
todo.txt	todo.txt upd	2 weeks ago

README License



Computational Analytical Mechanics

© 2024 Victor A. Bettachini

Access to the material: <https://unlam.github.io/ComputationalAnalyticalMechanics/>

This is the repository for the course [Computational Analytical Mechanics](#), for the [mechanical engineering degree](#) at the [Department of Engineering and Research in Technology](#), Universidad Nacional de La Matanza.

It's a code-based undergraduate course on analytical mechanics for engineering students with little to no prior programming knowledge. This 16-week flipped classroom course focuses on providing skills to calculate dynamics and strains of simple mechanical devices modelled as rigid bodies by solving Euler-Lagrange equations. Problems sets with increasingly complex exercises are introduced each week. Python-based solutions to previous ones are reused by students, who through small incremental modifications over the code presented by the teaching staff, build their own library of solutions to address mechanical modelling challenges.

Journal Open Source Education

[REVIEW]: Python-based Lagrange analytical mechanics course #293

Open



editorialbot opened on Jul 10

Collaborator ...

Assignees

emckiernan

Labels

review

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Submitting author: @bettachini (Bettachini, V. A.)
Repository: <https://github.com/unlam/ComputationalAnalyticalMechanics/>
Branch with paper.md (empty if default branch):
Version: v1.0.0
Editor: @emckiernan
Reviewers: @richmf, @laudavidovich
Archive: Pending
Paper kind: learning module

Status

JOSE Under Review

Status badge code:

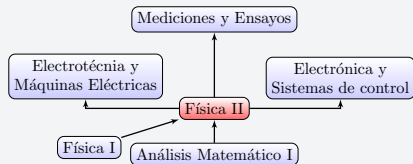
HTML: `<a href="https://jose.theoj.org/papers/722bf6629b6b6db72db8ecd6ed3cb19">img src="https://jose.theoj.org/papers/722bf6629b6b6db72db8ecd6ed3cb19/status.svg">`
Markdown: `[[status](https://jose.theoj.org/papers/722bf6629b6b6db72db8ecd6ed3cb19/status.svg)](https://jose.theoj.org/papers/722bf6629b6b6db72db8ecd6ed3cb19/status.svg)]`

Temario

- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica
- 4 UTN-FRA — Física II**
- 5 Para concluir

Cuadernos Jupyter solo como parte del curso

Física II en UTN-FRA

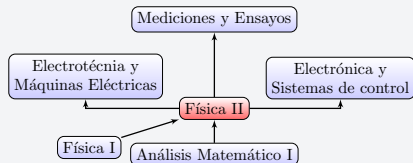


Salvando limitaciones de los alumnos

- No han cursado el análisis para estudiar distribuciones de carga eléctrica → Campo eléctrico aproximado con cargas discretas.

Cuadernos Jupyter solo como parte del curso

Física II en UTN-FRA

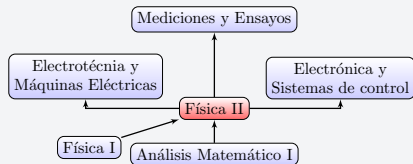


Salvando limitaciones de los alumnos

- No han cursado el análisis para estudiar distribuciones de carga eléctrica → Campo eléctrico aproximado con cargas discretas.
- Nulos conocimientos de programación y cálculo numérico → Funciones encapsuladas.

Cuadernos Jupyter solo como parte del curso

Física II en UTN-FRA

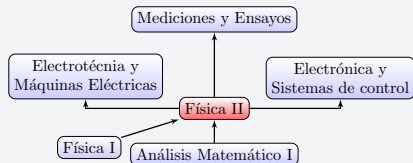


Salvando limitaciones de los alumnos

- No han cursado el análisis para estudiar distribuciones de carga eléctrica → Campo eléctrico aproximado con cargas discretas.
- Nulos conocimientos de programación y cálculo numérico → Funciones encapsuladas.
- El código permite analizar casos realistas a pesar de no dominar la matemática requerida.

Cuadernos Jupyter solo como parte del curso

Física II en UTN-FRA



Salvando limitaciones de los alumnos

- No han cursado el análisis para estudiar distribuciones de carga eléctrica → Campo eléctrico aproximado con cargas discretas.
- Nulos conocimientos de programación y cálculo numérico → Funciones encapsuladas.
- El código permite analizar casos realistas a pesar de no dominar la matemática requerida.
- Cuaderno Jupyter de ejercicios en formato de plantilla para completar.

Líneas de campo de anillo cargado

Posición de cargas

```
Q =  
[0.013, 0.200, 0.006, 0],  
[0.012, 0.199, 0.019, 0],  
[0.012, 0.198, 0.031, 0],  
[0.011, 0.195, 0.044, 0],  
[0.011, 0.192, 0.056, 0],  
[0.010, 0.188, 0.068, 0],  
[0.009, 0.184, 0.079, 0],  
#...  
[0.011, 0.192, -0.056, 0],  
[0.011, 0.195, -0.044, 0],  
[0.012, 0.198, -0.031, 0],  
[0.012, 0.199, -0.019, 0],  
[0.013, 0.200, -0.006, 0]]
```

✓ 0.0s

Python

Código gráficas

```
# Definimos una función para la densidad:  
def Lambda(t):  
    a = 1E-9  
    return a*cos(2*t)  
  
R = 20E-2 # Radio del anillo en m.  
N = 1000 # Número de cargas en que se divide el objeto.  
  
dt = 2*pi/N # Separación angular entre cargas.  
t0 = dt/2 # Angulo de la primera carga.  
Q = []  
ds = R*dt # Longitud de arco para cada segmento de anillo.  
for l in np.arange(N):  
    t = t0 + l*dt  
    dq = Lambda(t)*ds # Carga de un pequeño arco de anillo.  
    Q = Q + [[dq, R*cos(t0+l*dt), R*sin(t0+l*dt), 0]]
```

Python

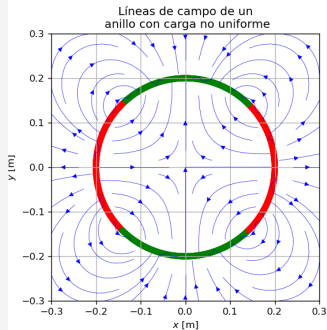
```
titulo = 'Líneas de campo de un\n anillo con carga  
no uniforme'
```

```
plotEf(Q, dx=0.3, title=titulo)
```

✓ 4.5s

Python

Líneas de campo



Saliendo del plano: dipolo eléctrico

Curvas equipotenciales

```
# Formato para la lista de cargas,  
# q en Coulombs, x,y,z en metros:  
# [  
# [q1, x1, y1, z1]  
# [q2, x2, y2, z2]  
# ]
```

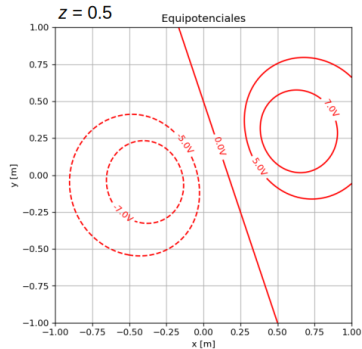
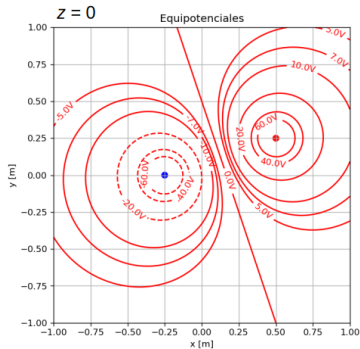
```
Q = [  
    [-1E-9, -0.25, 0, 0],  
    [ 1E-9, 0.5, 0.25, 0],  
]
```

```
niveles = [-60, -40, -20, -10, -7, -5, 0, 5, 7, 10,  
           20, 40, 60]
```

```
equipotencialesPuntuales(Q, niveles=niveles, z = 0)
```

✓ 0.1s

Python



Temario

- 1 Herramientas del aula
- 2 ¿Código en ingeniería mecánica?
- 3 UNLaM — Mecánica analítica
- 4 UTN-FRA — Física II
- 5 Para concluir

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.
- Reciclaje de código.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.
- Reciclaje de código.
- Docente: conferenciante → guía.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.
- Reciclaje de código.
- Docente: conferenciante → guía.
- Asistencia de IA.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.
- Reciclaje de código.
- Docente: conferenciante → guía.
- Asistencia de IA.
- Registro de entregas.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - El código es omnipresente en la industria mecánica.
- “No hay suficientes computadoras en el campus.”
 - La pandemia demostró que las hay: en hogares y trabajos.

Código en el aula

- Revalorización de la lectura.
- Reciclaje de código.
- Docente: conferenciante → guía.
- Asistencia de IA.
- Registro de entregas.
- Colaboración remota.

**MUCHAS
GRACIAS**

